

COMPSCI 546: Applied Information Retrieval - Spring 2026 ([website](#))

Assignment 6: Retrieval-Augmented Generation (Total: 100 points)

Description

This assignment consists of programming and analytical questions on Retrieval-Augmented Generation (RAG). You will build RAG pipelines using both sparse and dense retrieval, evaluate RAG systems and analyze the benefits and limitations of retrieval-enhanced approaches.

Instructions

- To start working on the assignment, you would first need to save the notebook to your local Google Drive. For this purpose, you can click on *Copy to Drive* button. You can alternatively click the *Share* button located at the top right corner and click on *Copy Link* under *Get Link* to get a link and copy this notebook to your Google Drive.
- For questions with descriptive answers, please replace the text in the cell which states "Enter your answer here!" with your answer. If you are using mathematical notation in your answers, please define the variables.
- You should implement all the functions yourself and should not use a library or tool for the computation, unless explicitly instructed to use one.
- For coding questions, you can add code where it says "enter code here" and execute the cell to print the output.
- **This assignment requires a GPU runtime.** In Colab, go to *Runtime -> Change runtime type* and select **T4 GPU**.
- To create the final pdf submission file, execute *Runtime->RunAll* from the menu to re-execute all the cells and then generate a PDF using *File->Print->Save as PDF*. Make sure that the generated PDF contains all the codes and printed outputs before submission.

Submission Details

- Due date: Thursday, April 23, 2026 at 11:59 PM (EDT).
- The final PDF file must be submitted to Gradescope.
- After copying this notebook to your Google Drive, please paste a link to it below. Use the same process given above to generate a link. ***You will not receive any credit if you don't paste the link!*** Make sure we can access the file.

LINK: <https://colab.research.google.com/drive/15eA-01fEEe92-cilgUj0GDZLYaxoZ5f?usp=sharing>

Academic Honesty

Please follow the guidelines under the *Collaboration and Help* section of the course website.

Setup and Data Download

We use the ANTIQUE dataset for this assignment, consistent with previous assignments. We will also use pre-trained transformer models for both retrieval and generation.

Please execute the cells below to install dependencies and download input files.

```
!pip install -q sentence-transformers faiss-cpu datasets gdown
transformers torch accelerate rouge-score bert-score

Preparing metadata (setup.py) ... 23.8/23.8 MB 43.0 MB/s eta
0:00:00
61.1/61.1 kB 3.6 MB/s eta
0:00:00

import os
import zipfile

# Download the ANTIQUE dataset files
!gdown 1lK8r5a_Aj9S4Cpd8k3x76ccBTluNguip -O HW07.zip

with zipfile.ZipFile('HW07.zip', 'r') as zip_file:
    zip_file.extractall('./')

if os.path.exists('HW07.zip'):
    os.remove('HW07.zip')

os.chdir('HW07')

# Setting input files
passage_file = "antique-collection.tok.clean_kstem"
test_queries_file = "antique-test-queries.tok.clean_kstem"
train_queries_file = "antique-train-queries.tok.clean_kstem"
val_queries_file = "antique-val-queries.tok.clean_kstem"
train_baseline_features_file = "train_baseline_features_top10"
val_baseline_features_file = "val_baseline_features_top10"
test_baseline_features_file = "test_baseline_features_top10"
stopwords_file = "stopword_INQUERY"

Downloading...
From (original): https://drive.google.com/uc?
id=1lK8r5a_Aj9S4Cpd8k3x76ccBTluNguip
```

```
From (redirected): https://drive.google.com/uc?
id=11K8r5a_Aj9S4Cpd8k3x76ccBTluNguip&confirm=t&uuid=38b02751-24e7-
457f-a28d-1105bb19b832
To: /content/HW07.zip
100% 33.3M/33.3M [00:01<00:00, 19.3MB/s]
```

1: Data Loading and Preparation (5 points)

We use files from the [ANTIQUE](#) dataset for this assignment. As described in the previous assignments, this is a passage retrieval dataset. The ANTIQUE dataset is a non-factoid question answering dataset where passages serve as answers to questions. We treat each query as a question and the highly relevant passages (relevance score ≥ 3) as gold answers.

The description of the input files provided for this assignment is given below.

Collection file

Each row of the file consists of the following information:

passage_id passage_text

The id and text information is tab separated. The passage text has been pre-processed to remove punctuation, tokenised and stemmed using the Krovetz stemmer.

Query files

You are provided with train, validation and test query files. Each row of the file consists of the following information:

query_id query_text

The id and text information is tab separated. The query text has been pre-processed to remove punctuation, tokenised and stemmed using the Krovetz stemmer.

Feature files

You are provided with train, validation and test feature files. Each row of the file consists of the following information:

query_id passage_id relevance_score vsm_score bm25_score

Each row contains features for a (query, passage) pair and is space separated. The *relevance_score* is the human annotated relevance score (1 to 4, where 4 is most relevant).

Stopwords file

The stopwords file contains the list of stopwords, one per line.

In the cell below, implement functions to load the collection, queries, stopwords, and feature files.

```

...
Load the passage collection.
Return:
    coll - dict mapping passage_id (str) to passage_text (str)
...
def loadCollection(passage_file):
    #enter your code here
    coll = {}
    with open(passage_file, 'r', encoding='utf-8') as file:
        for line in file:
            line = line.strip()
            if not line:
                continue

            passage_id, passage_text = line.strip().split('\t')
            coll[passage_id] = passage_text

    return coll
...
Load a query file.
Return:
    queries - dict mapping query_id (str) to query_text (str)
...
def loadQueryFile(filename):
    #enter your code here
    queries = {}
    with open(filename, 'r', encoding='utf-8') as file:
        for line in file:
            line = line.strip()
            if not line:
                continue

            qid, query_text = line.strip().split('\t')
            queries[qid] = query_text

    return queries
...
Load the stopwords into a set.
Return:
    stopwords - set of stopword strings
...
def loadStopWords(stopwords_file):
    #enter your code here
    stopwords = set([])
    with open(stopwords_file, 'r', encoding='utf-8') as file:
        for line in file:
            line = line.strip()
            if not line:
                continue

```

```

        stopword = line.strip()
        stopwords.add(stopword)

    return stopwords

'''
Parse a baseline features file and return relevance judgments.
Return:
    qrels - dict mapping query_id to a list of (passage_id,
    relevance_score) tuples
'''
def loadFeatureFile(features_file):
    #enter code here
    qrels = {}
    with open(features_file, 'r', encoding='utf-8') as file:
        for line in file:
            line = line.strip()
            if not line:
                continue

            qid, pid, rel_score, vsm_score, bm25_score =
line.strip().split()
            if qid not in qrels:
                qrels[qid] = []
            qrels[qid].append((pid, int(rel_score)))

    return qrels

coll = loadCollection(passage_file)
train_queries = loadQueryFile(train_queries_file)
val_queries = loadQueryFile(val_queries_file)
test_queries = loadQueryFile(test_queries_file)
stopwords = loadStopWords(stopwords_file)

train_qrels = loadFeatureFile(train_baseline_features_file)
val_qrels = loadFeatureFile(val_baseline_features_file)
test_qrels = loadFeatureFile(test_baseline_features_file)

print(f'Collection size: {len(coll)}')
print(f'Train queries: {len(train_queries)}')
print(f'Val queries: {len(val_queries)}')
print(f'Test queries: {len(test_queries)}')
print(f'Stopwords: {len(stopwords)}')
print(f'Test qrels: {len(test_qrels)} queries')

Collection size: 403492
Train queries: 2226
Val queries: 200

```

Test queries: 200
Stopwords: 418
Test qrels: 200 queries

2: BM25 Retrieval for RAG (20 points)

In this section, you will implement a BM25 retriever that can be used as the retrieval component of a RAG pipeline. Unlike previous assignments where BM25 scores were pre-computed, here you will implement BM25 from scratch over the full collection so that it can retrieve passages for **any** query.

2.1: Build an Inverted Index and Implement BM25 (15 points)

Build an inverted index over the passage collection and implement BM25 scoring.

Recall the BM25 scoring function:

$$\text{BM25}(q, p) = \sum_{t \in q} \text{IDF}(t) \cdot \frac{f(t, p) \cdot (k_1 + 1)}{f(t, p) + k_1 \cdot \left(1 - b + b \cdot \frac{|p|}{\text{avgdl}}\right)}$$

where:

- $f(t, p)$ is the term frequency of term t in passage p
- $|p|$ is the length of passage p (number of tokens)
- avgdl is the average passage length across the collection
- $k_1=1.2, b=0.75$
- $\text{IDF}(t) = \ln\left(\frac{N - n(t) + 0.5}{n(t) + 0.5} + 1\right)$, where N is the total number of passages and $n(t)$ is the number of passages containing term t

```
import math
from collections import defaultdict, Counter

class BM25Retriever:
    """
    A BM25 retriever that builds an inverted index over the collection
    and supports retrieval of top-k passages for a given query.

    Parameters:
        coll - dict mapping passage_id to passage_text
        k1 - BM25 parameter (default 1.2)
        b - BM25 parameter (default 0.75)
    """
    def __init__(self, coll, k1=1.2, b=0.75):
        self.k1 = k1
```

```

self.b = b
self.coll = coll
self.N = len(coll)

# TODO: Build the inverted index.
# You need to compute:
# self.index - dict mapping term -> dict of {passage_id:
term_frequency}
# self.doc_len - dict mapping passage_id -> number of tokens
# self.avgdl - average document length
# self.idf - dict mapping term -> IDF score

#enter code here
self.index = defaultdict(dict)
self.doc_len = {}
self.avgdl = 0
self.idf = {}

for pid, text in coll.items():
    tokens = text.split()
    self.doc_len[pid] = len(tokens)
    self.avgdl += len(tokens)

    for term in tokens:
        # save term to index (dict) and count them
        self.index[term][pid] = self.index[term].get(pid, 0) + 1

self.avgdl /= self.N

# dictionary mapping term to idf score
for term, postings in self.index.items():
    n = len(postings)
    self.idf[term] = math.log((self.N - n + 0.5) / (n + 0.5) +
1)

def score(self, query_terms, passage_id):
    """
    Compute the BM25 score for a query-passage pair.

    Parameters:
        query_terms - list of query terms
        passage_id - the passage to score
    Return:
        score - float, BM25 score
    """
    #enter code here
    score = 0
    dl = self.doc_len[passage_id]
    for term in query_terms:
        if term not in self.index or passage_id not in

```

```

self.index[term]:
    continue

    f = self.index[term][passage_id]
    numerator = f * (self.k1 + 1)
    denominator = f + self.k1 * (1 - self.b + self.b * dl /
self.avgdl)

    score += self.idf[term] * (numerator / denominator)

return score

def retrieve(self, query_text, k=10):
    """
    Retrieve the top-k passages for a given query.
    Only consider passages that contain at least one query term.

    Parameters:
        query_text - string, the query
        k - int, number of passages to retrieve
    Return:
        results - list of (passage_id, score) tuples, sorted by
score descending
    """
    #enter code here
    query_terms = query_text.split()

    candidate_passages = set()
    for term in query_terms:
        if term in self.index:
            candidate_passages.update(self.index[term].keys())

    results = []
    for pid in candidate_passages:
        s = self.score(query_terms, pid)
        if s > 0:
            results.append((pid, s))

    results.sort(key=lambda x: x[1], reverse=True)

    return results[:k]

print('Building BM25 index...')
bm25 = BM25Retriever(coll)
print(f'Index built. Vocabulary size: {len(bm25.index)}')
print(f'Average document length: {bm25.avgdl:.2f}')

# Test retrieval
sample_qid = list(test_queries.keys())[0]

```



```

sample_query = test_queries[sample_qid]
results = bm25.retrieve(sample_query, k=5)
print(f'\nSample query ({sample_qid}): {sample_query}')
print(f'Top 5 retrieved passages:')
for pid, s in results:
    print(f' {pid}: score={s:.4f}, text={coll[pid][:80]}...')

```

Building BM25 index...
 Index built. Vocabulary size: 149467
 Average document length: 41.12

Sample query (3990512): how can we get concentration on someth
 Top 5 retrieved passages:
 1128194_0: score=13.3215, text=its expect as the concentration of
 the molecule that have the scent they are try...
 3975076_2: score=13.2219, text=usually we get hiccup when our
 diaphragm is irritate what has work in the past v...
 4049238_1: score=13.0514, text=i really like that idea how can we
 get that on the ballot...
 1173778_0: score=12.8975, text=if the mole of hcl is unknow n u ll
 need a ph meter o something else to determine...
 17597_5: score=12.7034, text=pond life is a nice thing to call them
 they are no better than slime they need t...

2.2: Evaluate BM25 Retrieval (5 points)

Evaluate the BM25 retriever on the test queries using **NDCG@10** and **Recall@10**.

For **Recall@10**: the fraction of relevant passages (relevance ≥ 3) in the top-10 retrieved results out of all relevant passages for that query in `test_qrels`.

Note: Since BM25 retrieves from the full collection, the retrieved passages may or may not overlap with the candidate passages in `test_qrels`. For **NDCG@10**, assign a relevance score of 0 to any retrieved passage that does not appear in `test_qrels`.

Hint: Write `evaluateRetrieval` as a generic function that accepts any retriever with a `.retrieve()` method, so you can reuse it for dense and hybrid retrieval later.

```

import numpy as np
from sklearn.metrics import ndcg_score

```

```

...

```

Evaluate retrieval on test queries.

For each query in test_queries:

- 1. Retrieve top-10 passages using the retriever.*
- 2. Look up relevance scores from test_qrels. Passages not in test_qrels get score 0.*
- 3. Compute NDCG@10 and Recall@10.*

Parameters:

retriever - an object with a `.retrieve(query_text, k)` method
test_queries - dict of `query_id` -> `query_text`
test_qrels - dict of `query_id` -> list of (`passage_id`,
`relevance_score`)
k - number of results to retrieve (default 10)

Return:

mean_ndcg - float
mean_recall - float
...

```
def evaluateRetrieval(retriever, test_queries, test_qrels, k=10):
    recall_scores = []
    ndcg_scores = []

    for qid, query_text in test_queries.items():
        recall = 0
        passage_score = dict(test_qrels[qid])
        total_relevance = sum(1 for _, rel in test_qrels[qid] if rel
>= 3)

        retrieved_passages = retriever.retrieve(query_text, k)

        y_true = []
        y_score = []

        for pid, score in retrieved_passages:
            if pid not in passage_score:
                passage_score[pid] = 0

            rel = passage_score[pid]

            # calculate recall for a single query
            if rel >= 3:
                recall += 1

            # for ndcg
            y_true.append(rel)
            y_score.append(score)

        # calculate recall@k
        if total_relevance == 0:
            recall_scores.append(0)
        else:
            recall_scores.append(recall / total_relevance)

        # calculate ndcg@k
        if len(y_true) == 0:
            ndcg_scores.append(0)
        else:
```

```

        ndcg_scores.append(ndcg_score([y_true], [y_score], k=k))

mean_ndcg = np.mean(ndcg_scores)
mean_recall = np.mean(recall_scores)

return mean_ndcg, mean_recall

bm25_ndcg, bm25_recall = evaluateRetrieval(bm25, test_queries,
test_qrels)
print(f'BM25 Full-Collection Retrieval:')
print(f'  NDCG@10:    {bm25_ndcg:.4f}')
print(f'  Recall@10:   {bm25_recall:.4f}')

BM25 Full-Collection Retrieval:
  NDCG@10:    0.6131
  Recall@10:  0.4304

```

3: Dense Retrieval for RAG (20 points)

In this section, you will use a pre-trained bi-encoder model to build a dense retriever with FAISS indexing. This serves as an alternative retrieval backend for the RAG pipeline.

3.1: Build the Dense Retriever (15 points)

Use the pre-trained `all-MiniLM-L6-v2` sentence-transformer model (without fine-tuning) to encode all passages and build a FAISS flat inner product index. Then implement a retrieval function.

Important: Normalize all vectors to unit length so that inner product equals cosine similarity.

Steps:

1. Load the `all-MiniLM-L6-v2` model using `SentenceTransformer`.
2. Create an ordered list of (passage_id, passage_text) pairs from the collection.
3. Encode all passages using `model.encode()` with `normalize_embeddings=True`, `batch_size=256`, and `show_progress_bar=True`.
4. Build a FAISS `IndexFlatIP` index and add the passage embeddings.
5. Store the ordered list of passage_ids for mapping indices back.

```

import faiss
from sentence_transformers import SentenceTransformer

class DenseRetriever:
    ...
    A dense retriever using a pre-trained sentence-transformer and FAISS.

```

```

'''
def __init__(self, coll):
    #enter code here
    self.model = SentenceTransformer('all-MiniLM-L6-v2')

    # create ordered list
    items = list(coll.items())
    self.passage_ids = [pid for pid, text in items]
    self.passage_texts = [text for pid, text in items]

    # encode all passages
    self.embeddings = self.model.encode(
        self.passage_texts,
        normalize_embeddings=True,
        batch_size=256,
        show_progress_bar=True
    )

    # convert to FAISS-friendly type
    self.embeddings = np.array(self.embeddings, dtype='float32')

    # build IndexFlatIP
    dim = self.embeddings.shape[1]
    self.index = faiss.IndexFlatIP(dim)
    self.index.add(self.embeddings)

def retrieve(self, query_text, k=10):
    '''
    Retrieve the top-k passages for a given query.

    Steps:
    1. Encode the query with normalize_embeddings=True.
    2. Search the FAISS index for the top-k nearest neighbors.
    3. Map indices back to passage_ids.

    Parameters:
        query_text - string
        k - int
    Return:
        results - list of (passage_id, score) tuples
    '''
    #enter code here
    query_vec = self.model.encode(
        [query_text],
        normalize_embeddings=True
    )

    # convert to float32
    query_vec = np.array(query_vec, dtype='float32')

```

```

scores, indices = self.index.search(query_vec, k)

results = []
for idx, score in zip(indices[0], scores[0]):
    pid = self.passage_ids[idx]
    results.append((pid, float(score)))

return results

print('Building dense index (this may take a few minutes)...')
dense_retriever = DenseRetriever(coll)
print(f'Dense index built. Index size:
{dense_retriever.index.ntotal}')

# Test retrieval
results = dense_retriever.retrieve(sample_query, k=5)
print(f'\nSample query ({sample_qid}): {sample_query}')
print(f'Top 5 dense retrieved passages:')
for pid, s in results:
    print(f' {pid}: score={s:.4f}, text={coll[pid][:80]}...')

Building dense index (this may take a few minutes)...

{"model_id": "0c4aa4f74bbf47a3908e51a82b56435a", "version_major": 2, "version_minor": 0}

BertModel LOAD REPORT from: sentence-transformers/all-MiniLM-L6-v2
Key | Status | |
-----+-----+---
embeddings.position_ids | UNEXPECTED | |

Notes:
- UNEXPECTED :can be ignored when loading from different
task/architecture; not ok if you expect identical arch.

{"model_id": "8a90e5df8e1848429140ab82c4bade06", "version_major": 2, "version_minor": 0}

Dense index built. Index size: 403492

Sample query (3990512): how can we get concentration on someth
Top 5 dense retrieved passages:
248974_2: score=0.7211, text=with concentration you would do
something like this...
2036065_1: score=0.7001, text=just by concentration...
3265991_12: score=0.6923, text=concentration...
4087614_7: score=0.6899, text=try yoga for concentration...

```

```
1900286_7: score=0.6777, text=it might be hard for concentration you
will need to be extra patient understandi...
```

3.2: Evaluate Dense Retrieval (5 points)

Evaluate the dense retriever on the test queries using **NDCG@10** and **Recall@10**, following the same protocol as Section 2.2. You can reuse the `evaluateRetrieval` function.

```
dense_ndcg, dense_recall = evaluateRetrieval(dense_retriever,
test_queries, test_qrels)
print(f'Dense Retrieval (all-MiniLM-L6-v2):')
print(f'  NDCG@10:    {dense_ndcg:.4f}')
print(f'  Recall@10:   {dense_recall:.4f}')

print(f'\n{"Method":<35} {"NDCG@10":<12} {"Recall@10":<12}')
print('-' * 59)
print(f'{"BM25 Retrieval":<35} {bm25_ndcg:<12.4f}
{bm25_recall:<12.4f}')
print(f'{"Dense Retrieval (MiniLM)":<35} {dense_ndcg:<12.4f}
{dense_recall:<12.4f}')

Dense Retrieval (all-MiniLM-L6-v2):
  NDCG@10:    0.4781
  Recall@10:  0.3615
```

Method	NDCG@10	Recall@10
BM25 Retrieval	0.6131	0.4304
Dense Retrieval (MiniLM)	0.4781	0.3615

4: RAG Pipeline — In-Context Augmentation (45 points)

In this section, you will implement RAG pipelines that retrieve passages and use them as context for a language model to generate answers. This follows the **in-context augmentation** paradigm discussed in class, where retrieved passages are prepended to the query in the LM's input.

We will use the `google/gemma-2-2b-it` model as our language model — a 2.6B parameter instruction-tuned causal LM that fits on a T4 GPU in float16.

4.1: Load Gemma-2-2B-IT and Implement the RAG Pipeline (30 points)

Implement a RAG pipeline that:

1. Retrieves the top- k passages for a query using a given retriever.
2. Constructs a prompt that includes the retrieved passages as context.
3. Feeds the prompt to the language model to generate an answer.

Use the following prompt template:

Answer the following question based on the provided context. Give a detailed answer in 2-3 sentences. Do not reference passage numbers.

Context:

[1] {passage_1_text}

[2] {passage_2_text}

...

[k] {passage_k_text}

Question: {query_text}

Answer:

For the **closed-book** variant (no retrieval), use this prompt:

Answer the following question. Give a detailed answer in 2-3 sentences.

Question: {query_text}

Answer:

Important notes for Gemma (causal LM):

- Load using `AutoModelForCausalLM` with `torch_dtype=torch.float16` and `device_map="auto"`.
- When decoding, only decode the **newly generated tokens** (not the input prompt). Track the input length and slice the output accordingly.
- Use `max_new_tokens=200` and `do_sample=False` for generation.
- Set `tokenizer.pad_token = tokenizer.eos_token` if the pad token is not set.
- Log in to huggingface, accept the terms and add auth token so that you can access the model

```
!pip install -q huggingface_hub
from huggingface_hub import login
login()

{"model_id":"76a04116963742a4943c41292785974a","version_major":2,"version_minor":0}

from transformers import AutoModelForCausalLM, AutoTokenizer
import torch

MODEL_NAME = "google/gemma-2-2b-it"

# Load the model and tokenizer
tokenizer = AutoTokenizer.from_pretrained(MODEL_NAME)

if tokenizer.pad_token is None:
    tokenizer.pad_token = tokenizer.eos_token
```

```

lm_model = AutoModelForCausalLM.from_pretrained(
    MODEL_NAME,
    torch_dtype=torch.float16,
    device_map="auto"
)
# Use torch.float16 and device_map="auto" to fit on T4
#enter code here

device = next(lm_model.parameters()).device

print(f'Model loaded on {device}')

{"model_id":"851ee1689f834376ac7e2642e67d35f9","version_major":2,"version_minor":0}

{"model_id":"65f27ad03d4f4484b04a47800b0c803d","version_major":2,"version_minor":0}

{"model_id":"fdfc35d5a4734f2aba37160496b1f846","version_major":2,"version_minor":0}

{"model_id":"49b25f7bac8d4d79ba69c9af9af2de15","version_major":2,"version_minor":0}

`torch_dtype` is deprecated! Use `dtype` instead!

{"model_id":"ceff9326b4944f05a8918c2c63103522","version_major":2,"version_minor":0}

{"model_id":"9ab36f35c6524252b0202af628ae10f8","version_major":2,"version_minor":0}

{"model_id":"8b0cad251e8740449a5b54acae157e04","version_major":2,"version_minor":0}

{"model_id":"01de1ce4140148699f3e73806f0dd775","version_major":2,"version_minor":0}

{"model_id":"9991fe25a3cf4df29d1d43f92e4df3ba","version_major":2,"version_minor":0}

Model loaded on cuda:0

...

Build the RAG prompt from retrieved passages and a query.

Parameters:
    query_text - string, the question
    retrieved_passages - list of (passage_id, score) tuples
    coll - dict of passage_id -> passage_text

```



```

Return:
    prompt - string, the formatted prompt following the template above
    ...
def buildRAGPrompt(query_text, retrieved_passages, coll):
    #enter code here
    context_lines = []
    for i, (pid, score) in enumerate(retrieved_passages, start=1):
        context_lines.append(f'[{i}] {coll[pid]}')

    context_text = '\n'.join(context_lines)

    prompt = (
        "Answer the following question based on the provided context.
    "
        "Give a detailed answer in 2-3 sentences. Do not reference
    passage numbers.\n\n"
        "Context:\n"
        f"{context_text}\n\n"
        f"Question: {query_text}\n"
        "Answer:"
    )

    return prompt

    ...
Generate an answer using the causal language model given a prompt.

Parameters:
    prompt - string, the input prompt
    tokenizer - the tokenizer
    model - the causal LM
    device - torch device
    max_input_length - int, max tokens for input (default 1024)
    max_new_tokens - int, max new tokens to generate (default 200)

Return:
    answer - string, the generated answer (new tokens only, not the
    prompt)
    ...
def generateAnswer(prompt, tokenizer, model, device,
                  max_input_length=1024, max_new_tokens=200):
    #enter code here
    inputs = tokenizer(
        prompt,
        return_tensors="pt",
        truncation=True,
        max_length=max_input_length,
        padding=True
    )

```

```

inputs = {key: value.to(device) for key, value in inputs.items()}
input_length = inputs['input_ids'].shape[1]

outputs = model.generate(
    **inputs,
    max_new_tokens=max_new_tokens,
    do_sample=False,
    pad_token_id=tokenizer.pad_token_id
)

new_tokens = outputs[0][input_length:]
answer = tokenizer.decode(new_tokens,
skip_special_tokens=True).strip()

return answer

'''
Run the full RAG pipeline: retrieve, build prompt, generate answer.

Parameters:
    query_text - string
    retriever - an object with a .retrieve(query_text, k) method
    coll - dict of passage_id -> passage_text
    tokenizer - tokenizer
    model - language model
    device - torch device
    k - number of passages to retrieve (default 3)

Return:
    answer - string, the generated answer
    retrieved - list of (passage_id, score) tuples
'''
def ragPipeline(query_text, retriever, coll, tokenizer, model, device,
k=3):
    #enter code here

    retrieved = retriever.retrieve(query_text, k=k)
    prompt = buildRAGPrompt(query_text, retrieved, coll)
    answer = generateAnswer(prompt, tokenizer, model, device)

    return answer, retrieved

# Test the RAG pipeline with BM25 retriever
print('Testing RAG pipeline with BM25 retriever:\n')
for i, qid in enumerate(list(test_queries.keys())[:3]):
    query = test_queries[qid]

```

```

    answer, retrieved = ragPipeline(query, bm25, coll, tokenizer,
lm_model, device, k=3)
    print(f'Q ({qid}): {query}')
    print(f'A: {answer}')
    print(f'Retrieved: {[pid for pid, _ in retrieved]}')
    print()

```

Testing RAG pipeline with BM25 retriever:

Q (3990512): how can we get concentration onsometh

A: The provided context discusses how the concentration of a molecule can affect our perception of its presence. It explains that as the concentration decreases, our sense of smell becomes less reliable, but closer proximity allows for more detailed information about the scent.
Retrieved: ['1128194_0', '3975076_2', '4049238_1']

Q (714612): why do n t the water fall off earth if it s round

A: Gravity is holding the water down.
Retrieved: ['714612_0', '1369513_10', '714612_1']

Q (2528767): how do i determine the charge of the iron ion in fecl3

A: To determine the charge of the iron ion in FeCl₃, you need to consider the charge of the chlorine ions. Since each chlorine ion has a charge of -1, the iron ion must have a positive charge to balance the equation.
Retrieved: ['2528767_0', '2167806_1', '2528767_4']

4.2: Compare RAG Variants (15 points)

Now compare different RAG configurations:

1. **No retrieval (closed-book):** The LM answers the question without any context. Use the prompt: "Answer the following question: {query_text}\nAnswer:"
2. **BM25-RAG (k=1):** RAG with BM25 retriever, top-1 passage.
3. **BM25-RAG (k=3):** RAG with BM25 retriever, top-3 passages.
4. **BM25-RAG (k=5):** RAG with BM25 retriever, top-5 passages.
5. **Dense-RAG (k=3):** RAG with dense retriever, top-3 passages.

Evaluate each variant on the test queries using **ROUGE-L** as the evaluation metric. ROUGE-L measures the longest common subsequence between the generated answer and the gold answer(s). Please note that ROUGE-L is not necessarily an appropriate metric for evaluating long-form answers; the purpose of this assignment is to implement a full pipeline of retrieval, answer generation, and evaluation for educational purposes. Visit [https://en.wikipedia.org/wiki/ROUGE_\(metric\)](https://en.wikipedia.org/wiki/ROUGE_(metric)) to learn about ROUGE metrics.

For each test query, the gold answers are the passage texts with relevance score ≥ 3 in `test_qrels`. Compute ROUGE-L between the generated answer and each gold answer, and take the **maximum** score as the score for that query. If a query has no gold answers (no passage with relevance ≥ 3), skip it.

Note: To limit computation time, evaluate on a random sample of **50 test queries** that have at least one gold answer.

```
!pip install -q rouge-score

from rouge_score import rouge_scorer
import random

random.seed(42)

...
Evaluate a RAG configuration on a sample of test queries using ROUGE-L.

Steps:
1. Select test queries that have at least one gold answer (relevance >= 3).
2. Sample 50 queries from these.
3. For each query, run the RAG pipeline (or closed-book) to generate an answer.
4. Compute ROUGE-L F1 between the generated answer and each gold passage text.
5. Take the max ROUGE-L across gold passages for each query.
6. Return the mean of these max scores.

Parameters:
    retriever - a retriever object (or None for closed-book)
    test_queries - dict of query_id -> query_text
    test_qrels - dict of query_id -> list of (passage_id, relevance_score)
    coll - dict of passage_id -> passage_text
    tokenizer, model, device - LM components
    k - number of passages to retrieve (ignored if retriever is None)
    sample_size - number of queries to evaluate on (default 50)

Return:
    mean_rouge_l - float, mean max ROUGE-L F1 score
    ...

def evaluateRAG(retriever, test_queries, test_qrels, coll, tokenizer,
model, device,
                k=3, sample_size=50):
    # enter code here
    scorer = rouge_scorer.RougeScorer(['rougeL'], use_stemmer=True)

    # Step 1: Filter queries with at least one gold answer
    valid_qids = [
        qid for qid in test_queries if any(
            rel >= 3 for _, rel in test_qrels[qid])
    ]
```

[illegible]

```

print(f'BM25-RAG (k=1):    ROUGE-L = {bm25_k1_score:.4f}')

bm25_k3_score = evaluateRAG(bm25, test_queries, test_qrels, coll,
                             tokenizer, lm_model, device, k=3)
print(f'BM25-RAG (k=3):    ROUGE-L = {bm25_k3_score:.4f}')

bm25_k5_score = evaluateRAG(bm25, test_queries, test_qrels, coll,
                             tokenizer, lm_model, device, k=5)
print(f'BM25-RAG (k=5):    ROUGE-L = {bm25_k5_score:.4f}')

dense_k3_score = evaluateRAG(dense_retriever, test_queries,
                              test_qrels, coll,
                              tokenizer, lm_model, device, k=3)
print(f'Dense-RAG (k=3):    ROUGE-L = {dense_k3_score:.4f}')

# Print comparison table
print(f'\n{"Method":<25} {"ROUGE-L":<12}')
print('-' * 37)
print(f'{"Closed-book":<25} {closed_book_score:<12.4f}')
print(f'{"BM25-RAG (k=1)":<25} {bm25_k1_score:<12.4f}')
print(f'{"BM25-RAG (k=3)":<25} {bm25_k3_score:<12.4f}')
print(f'{"BM25-RAG (k=5)":<25} {bm25_k5_score:<12.4f}')
print(f'{"Dense-RAG (k=3)":<25} {dense_k3_score:<12.4f}')

```

Evaluating RAG variants (this may take several minutes)...

```

Closed-book:    ROUGE-L = 0.1702
BM25-RAG (k=1): ROUGE-L = 0.2077
BM25-RAG (k=3): ROUGE-L = 0.2384
BM25-RAG (k=5): ROUGE-L = 0.2223
Dense-RAG (k=3): ROUGE-L = 0.2392

```

Method	ROUGE-L
Closed-book	0.1702
BM25-RAG (k=1)	0.2077
BM25-RAG (k=3)	0.2384
BM25-RAG (k=5)	0.2223
Dense-RAG (k=3)	0.2392

5: Analysis and Discussion (10 points)

The lecture introduced the **Fusion-in-Decoder (FiD)** architecture for answer generation. Explain how FiD differs from the in-context augmentation approach you implemented in Section 4. What are the advantages of FiD over in-context augmentation? In what scenarios might in-context augmentation be preferred?

Answer In our implementation (in-context augmentation), the pipeline works as follows:

1. Retrieve relevant passages using a retriever (e.g., BM25 or dense retriever). Concatenate the retrieved passages together with the query into a single input prompt.
2. Feed this combined text into the language model, which encodes the entire sequence at once and generates the answer.
3. This approach performs early fusion, since all passages are merged into one sequence before being processed by the model.

In contrast, the Fusion-in-Decoder (FiD) architecture follows a different pipeline:

1. Retrieve relevant passages using a retriever.
2. For each passage, concatenate it with the query and encode each (query + passage) pair independently using the encoder.
3. Concatenate the resulting encoded representations (hidden states).
4. Use the decoder to generate the answer by attending over all encoded representations via cross-attention. This means FiD performs late fusion, where information from different passages is combined inside the decoder rather than in the input. ***The main advantage of FiD over in-context augmentation is that it preserves the structure of each passage. Since each passage is encoded separately, the model can better distinguish between relevant and irrelevant information and selectively attend to useful passages during generation. This avoids early mixing of tokens from different passages, reduces noise, and generally leads to better reasoning and answer quality. Additionally, FiD can scale better to multiple documents because it avoids the limitations of a single long input sequence.*** However, in-context augmentation may still be preferred in some scenarios. It is much simpler to implement, requires no architectural changes, and can be used directly with existing large language models (such as GPT-style models). It is also more computationally efficient, since it processes all text in a single pass rather than encoding each passage separately. Therefore, in-context augmentation is often preferred in practical systems where simplicity, efficiency, and compatibility with existing APIs are important.